

P1246R0

The `no_float` function attribute

Published Proposal, 2018-10-08

This version:

<http://wg21.link/p1246r0>

Issue Tracking:

[Inline In Spec](#)

Authors:

[Bruno Cardoso Lopes](#) (Apple)

[JF Bastien](#) (Apple)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/bcardosolopes/cpp-papers/blob/master/p1246r0.bs

Table of Contents

1 Introduction

2 Proposed Solution

References

Informative References

Issues Index

§ 1. Introduction

Code generation for floating-point is usually controlled by compiler flags. In [\[Clang\]](#) this can be achieved by invoking the compiler with `-mno-implicit-float`, `-msoft-float` or via other target specific mechanisms, for instance, compiling code for the [\[Darwin\]](#) kernel disables float by default.

This is desirable because:

- Some hardware doesn't support floating-point at all, requiring "soft-float" emulation of floating-point using integer instructions. This is inefficient and bloats the final binary and developers often want to opt-out of it.
- Some code wants to avoid saving and restoring floating-point registers, for example kernel syscalls don't want to trample over user-mode floating-point registers. Saving and restoring registers can get expensive: even with a conservative calling convention modern AVX512-enabled x86 CPUs have `ZMM` registers of 64 bytes each (in the worst case 32 such registers must be saved and restored).

This approach has many drawbacks:

- As is often the case with compiler flags, without changes to the Standard, each vendor ends up with a different mode, harming portability.
- Different pieces of a project might want different floating-point code generation; relying on extra compiler flags for specific translation units in a project is brittle and also has maintenance and portability costs. For instance, in an application one might want to opt out from floating-point only for specific pieces (e.g. cryptography code and kernel level `printf`-like utilities still want to use floating-point)
- Readability expectations: while maintaining and creating new code, the user has to look into the build system setup to find out whether the code in question can be translated to floating-point code. This might affect assumptions about rounding mode, auto vectorization and other implementation defined floating-point behavior.

§ 2. Proposed Solution

We propose a new function level attribute `no_float`. Attaching `no_float` to a function means:

1. Floating-point types cannot be used directly in the function.
2. The function's parameters and return types cannot be floating-point types.

3. Other functions that use floating-point types can be called from one marked with the `no_float` attribute as long as the callee's parameters and return type aren't floating-point.
4. No floating-point instructions should be emitted implicitly for the function. It's implementation defined what should happen instead, implementers might make it equivalent to what [\[GCC\]](#) and [\[Clang\]](#) do for `-msoft-float` (disable auto-vectorization, inlined `memcpy` cannot use SIMD registers, etc).

ISSUE 1 Alternatively for 3., we could make implementation defined what happens for such cross-calls, the compiler could in theory codegen two versions of the function, one with and another without and call the right one).

This attribute standardizes a combination of existing ad-hoc practices and documents code's expectation: functions shouldn't inadvertently use floating-point. Code fails to compile if the function uses floating-point, and the compiler knows to further avoid implicitly add floating-point register / instruction uses in the function.

Examples:

```
double h(double y) { return y + 0.3; }
double i(double y = 42.) { return y * 42.; }
double j() { return 0.2; }

[[no_float]] void g() {
    j(); // ERROR: j()'s return type is floating-point.
    h(a); // ERROR: h()'s parameter type is floating-point.
    i(); // ERROR: i()'s parameter type is floating-point.
}
```

We also propose adding module-level attributes for this, see [\[p1245r0\]](#).

§ References

§ Informative References

[Clang]

Clang: a C language family frontend for LLVM. URL: <http://clang.llvm.org>

[Darwin]

Darwin (operating system). URL: [https://en.wikipedia.org/wiki/Darwin_\(operating_system\)](https://en.wikipedia.org/wiki/Darwin_(operating_system))

[GCC]

[GCC, the GNU Compiler Collection](https://gcc.gnu.org). URL: <https://gcc.gnu.org>

[P1245R0]

Bruno Cardoso Lopes; JF Bastien. [export module containing \[\[attribute\]\]](http://wg21.link/p1245r0). URL: <http://wg21.link/p1245r0>

§ Issues Index

ISSUE 1 Alternatively for 3., we could make implementation defined what happens for such cross-calls, the compiler could in theory codegen two versions of the function, one with and another without and call the right one). [↩](#)